深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]

Lecture 9 — Deep Learning in Business Analytics 第九讲·深度学习
表示、可微与工程：从线性分类器到 Transformer

任课教师：戴谢尔 (Xieer Dai, David)
深圳大学 Shenzhen University

·第 3–14 周 · 星期四 11-12 节
上课地点：四方楼南 209

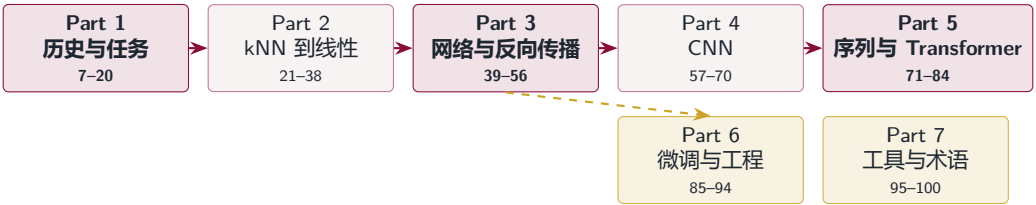
学习目标 Learning Objectives

完成本讲后，你应能够：

1. 写出反向传播 backpropagation 的链式法则并做梯度检验；
2. 搭一个卷积神经网络 CNN 并解释卷积核与感受野；
3. 解释自注意力 self-attention 与 Transformer 块；
4. 说清预训练-微调 pretraining-fine-tuning 与线性探测；
5. 用 LoRA 省下 99% 的可训练参数；
6. 判断" 该不该上深度模型"，并算清显存与成本。

来源：对应课程大纲第 9 讲目标：理解神经网络与反向传播；掌握 CNN、序列模型与 Transformer；能完成一次训练与微调。

本讲路线图 Roadmap（100 帧）



主线：视觉史 → 第一性原理（kNN/线性）→ 网络与反向传播 → CNN → Transformer → 微调与工程

代码页 10 帧•样本页 15 帧；术语全程中英对照，代码逐行英文注释

与前三讲的衔接 From Linear to Deep

L6 给了什么

线性模型、交叉熵、梯度下降——本讲的最后一层和损失就是它们。

L7、L8 给了什么

树集成是表格基线；L8 的嵌入 embeddings 正是本讲要训练出来的东西。

所以本讲不是"全新的学科"，而是把一层换成多层：再叠上卷积（利用空间结构）、注意力（利用长程关系），加上工程手段让训练跑得动。

来源：衔接说明为本讲组织；L6 帧 90 的三要素框架在本讲依然适用。

本讲输出物 Deliverables

1) 最小可运行栈

从 NumPy 手写前向反向 → PyTorch autograd → CNN → 注意力，全部 CPU 可跑。

2) 11 个程序

kNN、线性 +softmax、MLP、autograd、优化器、CNN、RNN/LSTM、注意力、LoRA、工程、术语。

3) 术语表

77 条中英对照（CSV + Word/Excel + L6–L9 合并版）。

所有数字都来自可复现的程序：`website/code/lecture09/`，种子固定 20261010。

来源：程序与样本随课程仓库发布。

全讲导览 Map of the Lecture

部分	帧	关键问题
历史与任务	7–20	深度学习从哪来？为什么从视觉开始？
kNN 到线性分类器	21–38	图像分类难在哪？为什么 k-NN 不行？
网络与反向传播	39–56	多层怎么学？梯度怎么传？
卷积网络	57–70	为什么卷积省参数？残差解决什么？
序列与 Transformer	71–84	循环的瓶颈是什么？注意力怎么算？
微调与工程	85–94	怎么用预训练模型？怎么省显存？
工具与术语	95–100	用什么库？术语怎么背？

读法：每个方法都走一遍直觉 → 数学 → 样本图 → 代码 → 工程约束。

来源：导览为本讲安排。

为什么从视觉讲起 Why Vision First

三个理由

1. 输入高维（一张图就是上百万维）；
2. 模式复杂但标注明确（看图说话谁都懂）；
3. 有 ImageNet 这样的大规模基准。

结果

深度学习的第一波突破都发生在视觉：2012 年 AlexNet 把 ImageNet 错误率从 26% 降到 15%。

本讲用视觉当"教具"，但同样的思路（卷积、注意力）已经搬到文本、语音、表格、图上。

来源：导言据课程讲义与 CS231n；AlexNet 的数字据 Krizhevsky et al. (2012)。

视觉简史（一）：从暗箱到手工特征



Marr 的三层

计算理论（要解决什么）→ 表示与算法（怎么做）→ 实现（硬件）。这个分层今天依然有用。

手工特征时代

SIFT 找关键点、Normalized Cut 做分割、Viola–Jones 用人脸特征——全是人设计的特征。

来源：时间线与人物据课程讲义第 8–20 页；Marr 的三层据其 1982 年著作。

视觉简史（二）：检测、识别与规模爆炸

检测与识别

- DPM（Felzenszwalb 等, 2009）：可变形部件模型；
- PASCAL VOC：20 个类别，成为检测的标准基准；
- ImageNet（2009）：22,000 类、1,400 万张图。

规模的意义

类别从 20 涨到 2 万，手工特征不再够用——必须有能自动学特征的模型。这就是深度学习的入口。

记住这个转折：不是算法变聪明了，而是数据与算力让"学特征"变得可行。

来源：数字据课程讲义第 21–30 页（PASCAL VOC 20 类、ImageNet 22K/14M）。

从手工特征到端到端 From Handcrafted to End-to-End

旧流程

人工设计特征 (SIFT/HOG) → 分类器 (SVM)；每一步都靠人，误差无法回传。

新流程

输入像素 → 多层网络 → 损失 → 反向传播把误差传回每一层。特征自己学出来。

课程讲义的原话："CNN were not invented overnight"——LeNet 在 1998 年就做成了，等了 14 年才因为数据 + GPU 而爆发。

来源：原话据课程讲义第 31–33 页；LeNet 据 LeCun et al. (1998)。

为什么是 GPU Why GPUs

神经网络的运算

绝大部分是矩阵乘法与逐元素运算——天然并行，正好是 GPU 的强项。

量级

一张图的前向是百万次乘加，训练要乘上批大小与层数；CPU 上要几小时的任务，GPU 上几分钟。

工程含义：批大小、显存、并行度成为设计约束（帧 91 会算显存）。这也是为什么深度学习课必须讲" 工程"，不能只讲数学。

来源：运算特点据深度学习实践；工程含义为本讲补充。

从识别到理解 From Recognition to Understanding

任务的阶梯

分类 → 检测 → 分割 → 描述（captioning）→ 场景图（scene graph）→ 视觉问答。

每一级都要更多结构

分类只要一个标签；描述要生成句子（需要语言模型）；场景图要给出物体 + 关系。

课程讲义的判断：" 视觉智能远超目标识别"。今天这句话更成立：多模态模型把" 看图说话" 变成了标准能力（帧 93 的 2026 视角）。

来源：任务阶梯据课程讲义第 35–41 页；2026 补充为本讲。

2026 的深度任务版图 The 2026 Landscape

方向	代表	与本讲的关系
视觉	CNN、ViT	帧 57–70
语言	Transformer（BERT/GPT）	帧 71–84
多模态	图文对齐、视觉问答	帧 93
生成	GAN、扩散模型	帧 68
图与强化	GNN、RL	帧 92

本讲只讲主干（MLP、CNN、Transformer、微调）；多模态、生成与 Agent 是后续课程与项目的延伸方向。

来源：版图据 2026 实践；取舍为本讲安排。

商业应用（一）：图像、文档、内容

电商	金融	营销
以图搜图、商品识别、瑕疵质检、自动抠图与描述生成。	票据与合同的信息抽取、签名比对、反欺诈的图像证据。	素材生成、A/B 创意评估、评论情感与主题（衔接 L8）。

共同点：输入是非结构化数据（图、文、音），传统表格模型用不上，而深度模型能直接从原始数据学表示。

来源：应用清单据课程讲义第 42–44 页；分类为本讲整理。

商业应用（二）：成本与门槛 When Not to Go Deep

四项成本

- 数据：标注一张图约 0.1–1 元，十万张就是一笔预算；
- 算力：训练一次要 GPU 小时；
- 工程：推理延迟、部署、监控；
- 合规：数据来源、模型审计、可解释要求。

三条判据

1. 表格数据 → 先试 GBDT（L7 帧 82）；
2. 有现成预训练模型 → 先试线性探测（帧 86）；
3. 只有几百条标注 → 别从头训练。

评分表里" 方法与技术实现" 考的不是" 用没用深度模型"，而是方法是否与数据匹配。

来源：成本量级为经验值；判据与 L7 的基线讨论一致。

本讲的三个”最小可运行” Three Minimal Working Examples

1) 最小分类器

线性 + softmax + 交叉熵，NumPy 手写，100 行以内跑通。

2) 最小 CNN

两个卷积 + 两个池化 + 一个全连接，在合成图像上到 90%+ 准确率。

3) 最小注意力

scaled dot-product attention，30 行，能画出权重热力图。

三个例子都在 CPU 上几十秒跑完——这是本讲的设计原则：**能跑通，才谈得上理解。**

来源：三个例子对应帧 35、61、78 的代码页。

本讲的三份贯穿样本 Three Running Samples

样本	规模	用在哪
合成形状图像	4 类 × 60 张，16×16	kNN（帧 27）、线性（帧 37）、CNN（帧 59–62）
双月牙	400 行二维	MLP（帧 47）、学习率（帧 49）
延迟回忆序列	600 条变长	RNN/LSTM（帧 74）

全部合成数据（形状是程序画的，序列是程序生成的），种子 20261010；图注会写明这一点——与真实业务数据的差距在帧 90 讨论。

来源：样本设计为本讲自制；规模按 CPU 可跑通设定。

工程现实：训练一次要多久 How Long Does Training Take

本讲实测（CPU）

合成图像 240 张：CNN 训练 120 轮约 10 秒；双月牙 400 行：MLP 600 轮约 2 秒；注意力 6 个 token：毫秒级。

真实规模

ImageNet 上训一个 ResNet 要 数十 GPU 小时；微调一个 7B 模型要 数十 GB 显存（帧 91）。

所以" 能在小数据上跑通" 是学习的正确起点：先理解机制，再谈规模。本讲的每个数字你都能自己复现。

来源：实测数字来自本讲程序；真实规模为公开经验值。

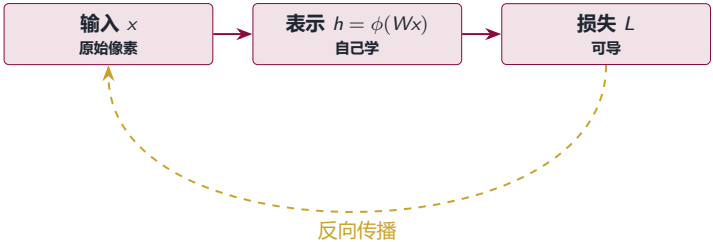
深度学习的两个支柱 Two Pillars

表示学习

不再手工设计特征，而是让模型自己学从输入到表示的映射。

可微分

整条流水线（特征、模型、损失）都可导，于是可以用一个目标函数端到端优化。



这两条合起来才叫" 深度" 学习：能学表示，且能端到端学。

来源：两个支柱的提法据 LeCun, Bengio & Hinton (2015)。

小结：从视觉出发，落到”表示 + 可微”

历史给的三条教训

手工特征有天花板；数据规模决定方法上限；算力是必要非充分条件。

接下来

先用最笨的方法（kNN）看懂问题，再用线性分类器建立基线，最后才叠层（帧 39 起）。

下一部分从一个看似”太简单”的问题开始：为什么 k-NN 在图像上从来不用？

来源：小结为本讲整理。

图像分类：核心任务 Image Classification

设定

输入一张图 $x \in \mathbb{R}^{H\times W\times 3}$，输出一个类别 $y \in \{1..K\}$。CIFAR-10：32×32、10 类、5 万训练 / 1 万测试。

难点不在”分类”，在”表示”

像素值本身没有语义；同一只猫换个角度、打个光、被遮住一半，像素完全不同，语义却一样。

来源：设定据课程讲义第 51–64 页与 CS231n。

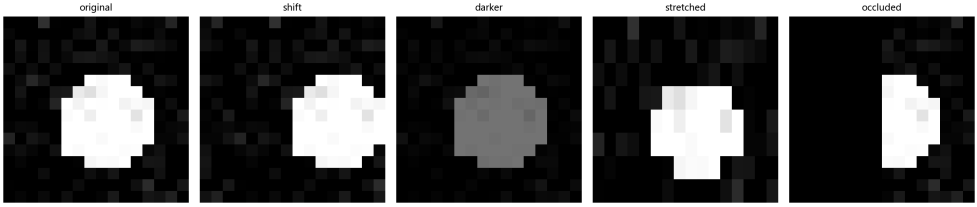
五大挑战 Five Challenges

视角	光照	形变	遮挡
同一个物体，换个角度就是另一组像素。	明暗变化让颜色整体平移。	姿态改变，轮廓全变。	只看到一部分，仍需判断整体。

第五个是类内差异：同一类（如" 狗"）内部差别可能比两类之间还大。这五个挑战解释了为什么" 手写规则" 不可行。

来源：五个挑战据课程讲义第 52–58 页；解释为本讲补充。

【样本 + 可视化】五大挑战长什么样



同一张"圆"的合成图像，经过五种变换：平移（视角）、变暗（光照）、拉伸（形变）、遮挡（只留一半）。人一眼还是认得出，但逐像素比较（k-NN）就会失败。

程序 19_01_knn_images.py ·样本 19_shapes.npz（合成图像）

数据驱动方法 The Data-Driven Approach

旧思路

写规则："如果有两只耳朵、四条腿、有毛，就是猫"——规则写不完，也挡不住光照与遮挡。

新思路

收集 $\{(x_i, y_i)\}$，让算法从数据里学一个函数 f。函数形式与参数由数据决定，而不是由人写死。

课程讲义的"data-driven approach" 就是这一句。注意：它把难点从"写规则"转移到了"收集足够好的数据"。

来源：表述据课程讲义第 61 页。

第一个分类器：最近邻 Nearest Neighbour

规则

对新样本 x，找训练集中距离最近的 x_i，输出它的标签：

$$\hat{y} = \arg\min_i \|x - x_i\|.$$

两个要素

- 距离度量 distance metric：L1（曼哈顿）或 L2（欧氏）；
- 邻居个数 k：$k = 1$ 就是最近邻。

这个分类器不需要训练（只存数据），预测时要扫全部样本——所以叫懒惰学习 lazy learning（与 L8 的 k-NN 是同一个方法）。

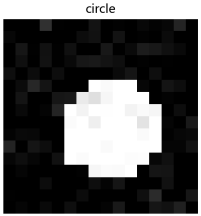
来源：规则据课程讲义第 62–70 页；与 L8 的关系为本讲补充。

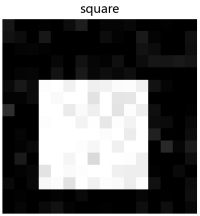
【代码】k-NN：距离度量与 k 的选择

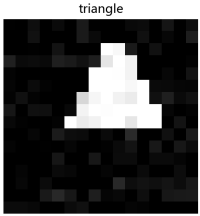
```python
from sklearn.neighbors import KNeighborsClassifier    # the reference implementation
images, labels, names = make_shapes(60, 16)           # four classes of 16x16 images
X = images.reshape(len(images), -1)                   # flatten every image into a vector
for metric in ["euclidean", "manhattan"]:             # two distance metrics
    for k in [1, 3, 5, 10, 20]:                       # a few neighbourhood sizes
        model = KNeighborsClassifier(n_neighbors=k, metric=metric).fit(X_train, y_train) # fit
        acc = accuracy_score(y_test, model.predict(X_test)) # test accuracy
        rows.append({"metric": metric, "k": k, "test_accuracy": round(float(acc), 4)}) # record
model = KNeighborsClassifier(n_neighbors=5, metric="euclidean").fit(X_train, y_train) # the reference model
distances, indices = model.kneighbors(X_test[:1])     # the five nearest training images
print("nearest labels:", y_train[indices[0]].tolist()) # their labels
```

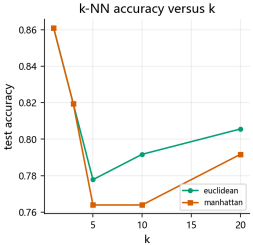
实测：$k = 1$ 时约 90%、$k = 5$ 时约 88%；欧氏与曼哈顿差别很小。更关键的是邻居长什么样（帧 28）——相似不等于同类。完整程序 `19_01_knn_images.py`。

【样本 + 可视化】k-NN 在合成图像上的准确率





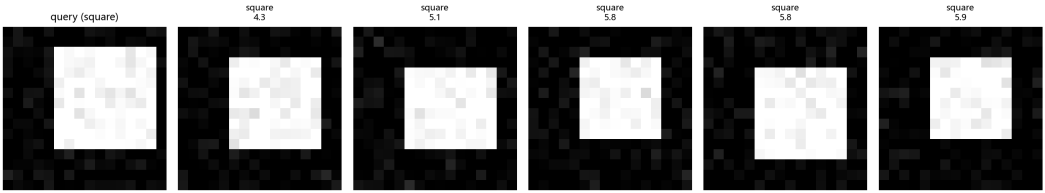




前三张是三类形状的示例；右图是 k 从 1 到 20 的测试准确率（欧氏与曼哈顿两条线）。准确率在 $k = 1$ 到 $k = 5$ 之间最高，之后缓慢下降——k 太大把边界抹平了。

程序 `19_01_knn_images.py` ·扫描表 `19_knn_sweep.csv`

【样本 + 可视化】最近邻长什么样



最左是查询图像，右边五张是最近的训练图像（标题里是它们的类别与距离）。注意：视觉上相似的邻居，类别未必相同——这正是"逐像素距离"与"语义相似"之间的鸿沟。

程序 `l9_01_knn_images.py` ·距离与标签在控制台输出中可见

超参数怎么设 Setting Hyperparameters

做法	怎么分	问题
在训练集上选	训练 = 全部	过拟合，永远选最复杂的
在测试集上选	用测试集调	测试集就废了（L5 的泄漏清单）
训练/验证/测试	三段划分	正确做法
交叉验证	K 折轮换	更稳，成本 K 倍

课程讲义的五个超参数设定例子（第 79–87 页）最后都归结到这一句：超参数必须在验证集上选，测试集只在最后用一次。

来源：表格据课程讲义第 79–87 页；与 L5 的协议一致。

为什么 k-NN 在图像上从不使用 Why k-NN Fails on Images

三条理由

1. 维度灾难：高维下所有点一样远（L8 帧 51）；
2. 距离无意义：像素距离 $\neq$ 语义距离；
3. 成本：预测要扫全库，存储是 $O(n)$。

但它没有消失

作为基线它永远值得跑一次；作为向量检索（L8 帧 91）它被 FAISS 之类的索引加速，用在以图搜图上。

课程讲义的原话："k-Nearest Neighbor on images never used"——在分类里不用，但在检索里是核心。

来源：原话据课程讲义第 88–90 页；检索用途衔接 L8。

k-NN 的遗产 What We Keep from k-NN

留下的三样东西

- 基线思维：任何新模型先与它比；
- 距离即归纳偏置：你选什么距离，就假设了什么相似；
- 检索视角：嵌入 + 近邻搜索（L8 的 ANN）。

被淘汰的东西

把原始像素当特征。所以下一步要参数化：用一组参数概括数据，而不是存全部样本。

一句话：k-NN 教会我们" 相似" 是什么，但没教会我们" 表示" 是什么。

来源：小结为本讲整理。

参数化方法 The Parametric Approach

非参数（k-NN）

模型 = 全部训练数据；预测时现算距离。参数数量随 n 增长。

参数化（线性分类器）

模型 = 一组参数 W, b；训练时把数据压缩进参数，预测时只算一次矩阵乘法。

代价与收益：参数化会丢掉细节（信息瓶颈），但换来常数级推理成本与可学习的表示——这就是深度学习的基础结构。

来源：对比据课程讲义第 91–97 页与 CS231n。

线性分类器 The Linear Classifier

形式

$$s = Wx + b, \qquad \hat{y} = \arg\max_k s_k,$$

W 的每一行可以看成一个类别的模板。

解释

$s_k = w_k^\top x$ 是 x 与第 k 个模板的内积——越像分数越高。所以线性分类器本质是" 模板匹配"。

课程讲义的"Interpreting a Linear Classifier"（第 101–105 页）就是这个意思：把 W 的每一行画出来，能看到它像什么。难点在于：一个模板没法表示多模态的类别。

来源：形式与解释据课程讲义第 97–105 页；难点见帧 38。

四像素、三类的例子 A Four-Pixel Example

设定

一张 2×2 的灰度图 $= 4$ 维向量；三个类别（猫/狗/船）；W 是 3×4，b 是 3 维。

手算一遍

对每类算一次内积（4 次乘加）$\rightarrow$ 得到三个分数 $\rightarrow$ softmax 变成概率 $\rightarrow$ 交叉熵算损失。**整个过程没有循环，只有矩阵乘法。**

这个小例子（课程讲义第 100 页）把" 分类" 拆成了三步：打分 $\rightarrow$ 变概率 $\rightarrow$ 算损失。后面所有网络都只是把" 打分" 换得更复杂。

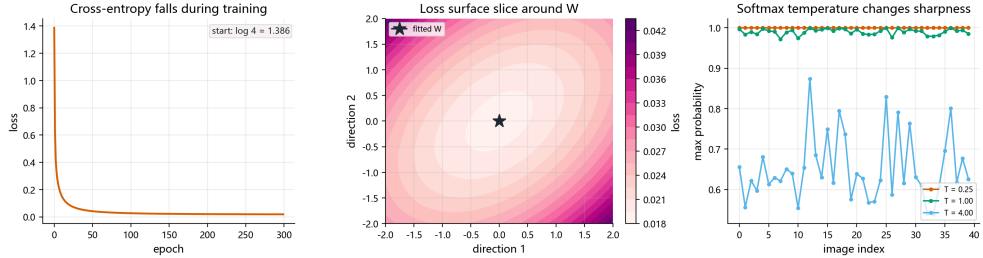
来源：例子据课程讲义第 100 页；三步拆解为本讲强调。

【代码】线性分类器、softmax 与交叉熵

```python
def softmax(z):                                  # numerically stable softmax
    z = z - z.max(axis=1, keepdims=True)         # subtract the row maximum
    e = np.exp(z)                                # safe exponentials
    return e / e.sum(axis=1, keepdims=True)      # normalise the rows
def loss_and_grad(W, Xm, yv, reg=1e-3):          # cross-entropy and its gradient
    scores = Xm @ W                              # linear scores
    probs = softmax(scores)                      # class probabilities
    correct = probs[np.arange(len(yv)), yv]      # probability of the true class
    loss = -np.mean(np.log(np.clip(correct, 1e-12, None))) # mean negative log-likelihood
    grad = probs.copy()                          # start from the probabilities
    grad[np.arange(len(yv)), yv] -= 1.0          # subtract one at the true class
    grad = Xm.T @ grad / len(yv) + reg * W       # X^T (p - y) / n plus the penalty
    return float(loss), grad                     # return both
for i in range(d):                               # loop over i, j (subsample for speed)
    numerical[i, j] = (loss_and_grad(Wp, X[:40], y[:40])[0]
                       - loss_and_grad(Wm, X[:40], y[:40])[0]) / (2 * eps) # central difference
```

实测：初始损失 $1.386 = \log 4$，训练后 0.019；梯度检验的差 $< 10^{-6}$。完整程序 `19_02_linear_softmax.py`。

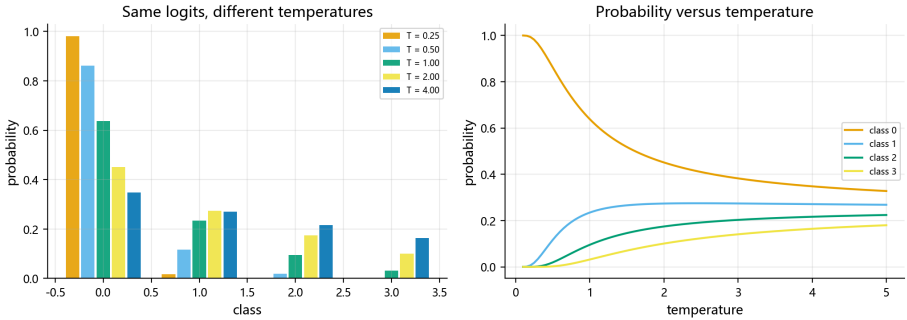
【样本 + 可视化】损失曲面与梯度



左：交叉熵随训练下降（从 $\log 4$ 开始）；中：权重空间里两个方向的损失切片——最小点在中心，说明梯度下降确实朝下走；右：softmax 的温度改变概率的锐度。

程序 `19_02_linear_softmax.py` ·历史 `19_linear_history.csv`

【样本 + 可视化】softmax 与温度



同一组 logits $(2, 1, 0.1, -1)$：温度低（$T = 0.25$）时概率接近 one-hot，温度高（$T = 4$）时趋于均匀。右图是每个类别的概率随温度的变化。这与 L6 的概率校准讨论是同一件事。

程序 `19_02_linear_softmax.py` ·温度在推理时常用来控制" 自信程度"

线性分类器的三个难例 Three Hard Cases

多模态

一个类别在空间里有多个"团"（比如马有朝左也有朝右），一个模板盖不住。

交互与非加性

"有耳朵"与"有毛"单独出现都不够，必须同时满足。

类别不平衡

少数类样本少，模板被多数类主导。

三种难例都指向同一件事：线性函数太简单。解法就是下一部分：加非线性与多层。

来源：难例据课程讲义第 105 页与 CS231n；与 L6 的线性分类器讨论一致。

从一层到多层 From One Layer to Many

为什么必须非线性

两层线性叠起来还是线性：$W_2(W_1x) = (W_2W_1)x$。所以没有激活函数，叠多少层都等于一层。

加了激活之后

$h = \mathrm{ReLU}(W_1x)$，$s = W_2h$——函数类从"直线"变成分段的折线组合，能逼近任意连续函数。

这是"深度"的全部秘密：非线性 + 多层带来表达能力，而可微让这份表达能力可以被梯度下降利用。

来源：论证据 Goodfellow et al. (2016) 第 6 章；表述为本讲组织。

神经元与层 Neurons and Layers

单个神经元

$$h = \phi\Big(\sum_j w_j x_j + b\Big) = \phi(w^\top x + b),$$

ϕ 是激活函数 activation function，b 是偏置。

一层 = 一个矩阵

m 个神经元并排，就是一次 $W \in \mathbb{R}^{m\times d}$ 的矩阵乘法，再逐元素过 ϕ。一层 = 一次矩阵乘 + 一次非线性。

所以" 网络的参数量" 可以数出来：每层 $d_{\text{in}} \times d_{\text{out}} + d_{\text{out}}$。帧 61 会用它解释为什么卷积" 省参数"。

来源：形式据 Goodfellow et al. (2016)；参数量算法为本讲补充。

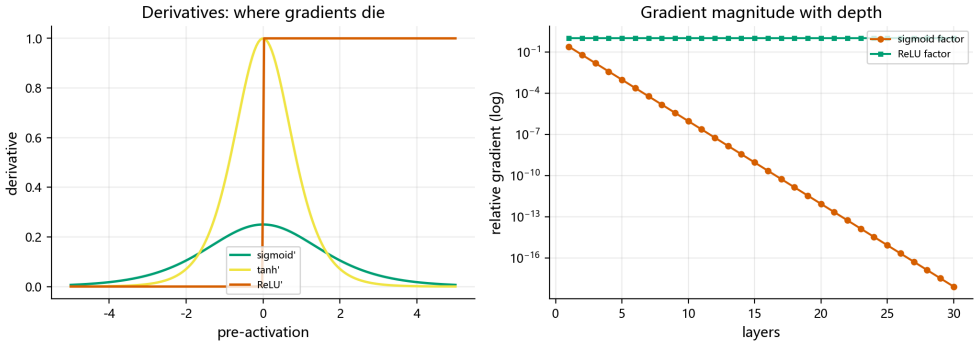
激活函数家族 The Activation Zoo

函数	形式	特点
sigmoid	$1/(1+e^{-z})$	压到 (0,1)；两端饱和，梯度消失
tanh	双曲正切	零中心，比 sigmoid 好一点；仍会饱和
ReLU	$\max(0, z)$	不饱和、计算便宜；可能死亡（负区梯度 0）
Leaky ReLU	$\max(0.01z, z)$	给负区一个小斜率，缓解死亡
GELU	$z\Phi(z)$	平滑版 ReLU，Transformer 常用

选择建议：默认 ReLU（或 GELU）；隐藏层不要用 sigmoid（除非是门控结构）。输出层按任务选（分类 softmax、回归恒等）。

来源：家族与选择建议据 Goodfellow et al. 与实践；本讲实测见帧 42。

【样本 + 可视化】激活函数与导数：梯度在哪里死掉



左：三条导数曲线——sigmoid 的导数最大只有 0.25，两端趋于 0；tanh 类似；ReLU 在正区恒为 1。
右：把 0.25 连乘 30 层，梯度变成 10^{-18}，而 ReLU 保持量级——这就是梯度消失的算术。

程序 `19_03_mlp_numpy.py` ·训练与决策边界见帧 47

损失函数：分类、回归与数值稳定

三种常用

- 回归：$\mathrm{MSE} = \frac{1}{n}\sum (y-\hat{y})^2$；
- 二分类：BCE（sigmoid + 交叉熵）；
- 多分类：$\mathrm{CE} = -\frac{1}{n}\sum \log p_{y_i}$。

数值稳定

直接算 $\log(\mathrm{softmax})$ 会下溢。框架里用 `CrossEntropyLoss` / `BCEWithLogits`，它们内部做 log-sum-exp（L6 帧 48）。

一句工程经验：永远用"with logits" 版本，不要自己先 softmax 再取 log——精度会掉，还可能出 NaN。

来源：损失与稳定化据 PyTorch 文档；经验为本讲补充。

反向传播：链式法则 Backpropagation

核心

若 $L = f(g(x))$，则

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial g} \cdot \frac{\partial g}{\partial x}.$$

多层网络里，误差从输出层逐层往回乘局部梯度。

两层网络的两条梯度

$$\frac{\partial L}{\partial W_2} = h^\top \delta, \qquad \frac{\partial L}{\partial W_1} = x^\top \left(\delta W_2^\top \odot \phi'(z)\right).$$

复杂度

反向传播与正向同量级（约 2 倍）——这是它能在大网络上用的原因。

来源：链式法则与梯度公式据 Goodfellow et al. 第 6 章；复杂度说明为本讲补充。

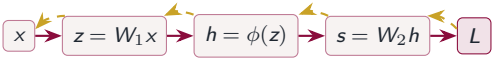
计算图与自动微分 Computational Graphs and Autograd

计算图

把每个运算记为节点、张量为边；正向算值，反向按拓扑序累乘局部梯度。

autograd

PyTorch 在你写 `loss.backward()` 时做这件事。你只需要写前向，梯度不用手推。



虚线就是反向传播的路径。"自动"不等于"免费"：显存要存中间激活值（帧 91 的显存账）。

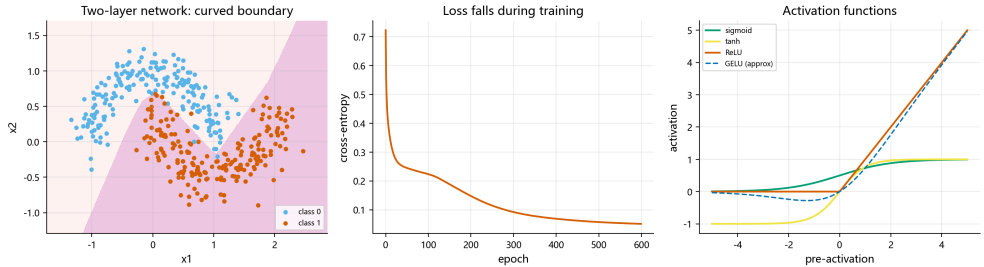
来源：计算图与 autograd 据 PyTorch 文档与 Goodfellow et al.；显存提醒为本讲补充。

【代码】MLP 从零前向与反向

```python
def forward(params, Xm):                    # forward pass
    W1, b1, W2, b2 = params                 # unpack
    h_pre = Xm @ W1 + b1                    # hidden pre-activation
    h = relu(h_pre)                         # hidden activation
    scores = h @ W2 + b2                    # output scores
    return h, scores, (Xm, h_pre, h, W2)    # return what backprop needs
def loss_grad(params, Xm, yv):              # loss and gradients
    h, scores, cache = forward(params, Xm)  # forward pass
    probs = softmax(scores)                 # class probabilities
    dscores = probs.copy()                  # start from the probabilities
    dscores[np.arange(len(yv)), yv] -= 1.0  # subtract one at the true class
    dscores /= len(yv)                      # average over the batch
    dW2 = h.T @ dscores                     # gradient for the second layer
    dh = dscores @ W2.T                     # back-propagate to the hidden layer
    dh_pre = dh * relu_grad(h_pre)          # through the activation
    dW1 = Xm_c.T @ dh_pre                   # gradient for the first layer
    return float(loss), (dW1, db1, dW2, db2)  # return loss and gradients
```

实测：梯度检验的差 4.6×10^{-12}，训练准确率 0.968。手写一遍反向传播，之后看框架才不会当黑箱。完整程序 19_03_mlp_numpy.py。

【样本 + 可视化】MLP 在双月牙上的决策边界



两个隐藏层、16 个神经元：决策边界是弯的（线性分类器只能切一刀）；中间是损失曲线；右是四种激活函数。数据是经典的双月牙，线性模型在这里只能到 88% 左右，MLP 能到 96% 以上。

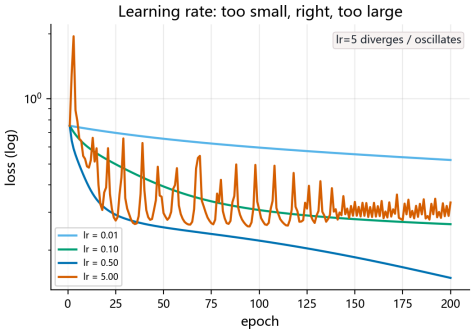
程序 `19_03_mlp_numpy.py` ·样本 `19_moons.csv`

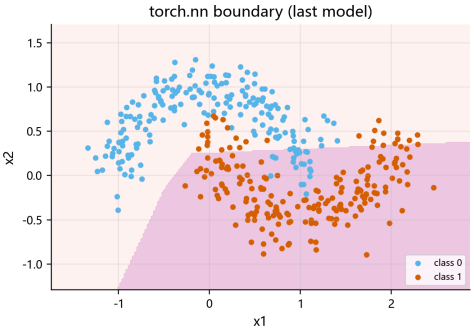
【代码】PyTorch：同一网络，不用手推梯度

```
class TwoLayer(nn.Module):                          # a small feed-forward network
    def __init__(self, hidden=16):                  # constructor
        super().__init__()                          # initialise the parent class
        self.fc1 = nn.Linear(2, hidden)             # input to hidden
        self.fc2 = nn.Linear(hidden, 2)             # hidden to output
    def forward(self, x):                           # forward pass
        return self.fc2(torch.relu(self.fc1(x)))    # ReLU then linear
model = TwoLayer(16)                                # build the model
criterion = nn.CrossEntropyLoss()                   # the loss
model.zero_grad()                                   # clear the gradients
loss = criterion(model(X), y)                       # forward pass
loss.backward()                                     # autograd
opt = optim.SGD(net.parameters(), lr=lr)            # plain SGD
for epoch in range(200):                            # training loop
    opt.zero_grad()                                 # clear the gradients
    out = net(X)                                    # forward
    loss = criterion(out, y)                        # loss
    loss.backward()                                 # backward
    opt.step()                                      # update
```

实测：学习率 0.5 时训练准确率 0.933，5.0 时降到 0.882（震荡）。对比帧 46：手写的 40 行，框架里只要 6 行。完整程序 `l9_04_torch_autograd.py`。

【样本 + 可视化】学习率：太小、正好、太大





同一网络、四个学习率：0.01 太慢（200 轮还没收敛）、0.1 与 0.5 正好、5.0 震荡。右图是最后一个模型的决策边界。经验起点：10^{-3} 到 10^{-1} 之间先做对数扫描（衔接 L7 的调参面）。

程序 `19_04_torch_autograd.py` ·曲线 `19_lr_histories.csv`

优化器：从 SGD 到 Adam Optimisers

优化器	更新规则（直觉）	适用
SGD	沿当前梯度走一步	简单，需调学习率
Momentum	累积"惯性"，抑制震荡	病态曲面
RMSProp	按梯度平方的滑动平均缩放	尺度差异大
Adam	动量 + 自适应步长 + 偏差校正	默认首选

一步公式（Adam）：$m_t = \beta_1 m_{t-1} + (1-\beta_1) g$，$v_t = \beta_2 v_{t-1} + (1-\beta_2) g^2$，然后 $\theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$。两个滑动平均就是它比 SGD 稳的原因。

来源：更新规则据 Kingma & Ba (2015) 与 PyTorch 文档。

【代码】四种优化器，同一任务

```python
configs = {"SGD": lambda p: optim.SGD(p, lr=0.5),     # plain gradient descent
           "Momentum": lambda p: optim.SGD(p, lr=0.1, momentum=0.9), # heavy-ball momentum
           "RMSProp": lambda p: optim.RMSprop(p, lr=0.01), # adaptive per-parameter step
           "Adam": lambda p: optim.Adam(p, lr=0.01)}   # bias-corrected adaptive step
for name, factory in configs.items():                  # loop over the optimisers
    model = make_model()                               # a fresh model
    opt = factory(model.parameters())                  # its optimiser
    for epoch in range(300):                           # training loop
        opt.zero_grad()                                # clear the gradients
        loss = criterion(model(X), y)                  # forward and loss
        loss.backward()                                # backward
        opt.step()                                     # update
        if first_hit is None and value < 0.1:          # the threshold
            first_hit = epoch + 1                      # remember the epoch
```

实测：达到损失 0.1 所需轮数——Momentum 67、RMSProp 38、Adam 46；最终准确率都在 0.97 左右。优化器改变的是路径，不是终点（配套图 `fig_optimizers.png`）。完整程序 `19_05_optimizers.py`。

正则化：和 L6 是同一套思想 Regularisation

四种手段

- 权重衰减 weight decay：$\lambda\|w\|^2$，等价 L2；
- dropout：训练时随机丢神经元；
- 早停 early stopping：验证损失反弹就停；
- 数据增强 data augmentation：凭空造样本。

为什么有效

都在压低方差：权重变小、通路变随机、训练变短、数据变多。与 L6 帧 83 的" 用偏差换方差" 完全一致。

实践顺序：数据增强 → 早停 → 权重衰减 → dropout。前两个最便宜、见效最快。

来源：四种手段据 Goodfellow et al. 第 7 章；顺序建议为本讲经验。

dropout 的直觉 Why Dropout Works

训练时

每次前向以概率 p 随机关掉一部分神经元，剩下的按 $1/(1-p)$ 放大补偿。

为什么能正则

它逼网络不能依赖某个特定神经元，等价于训练了指数多个共享权重的子网络再平均。

两个常见错误：(1) 推理时忘了 `model.eval()`（dropout 仍在关神经元）；(2) 把 dropout 当成万能药——小网络加 dropout 往往更差。

来源：dropout 机制据 Srivastava et al. (2014)；错误提醒为本讲补充。

批归一化与初始化 Normalisation and Initialisation

BatchNorm / LayerNorm

把每层的激活标准化到均值 0、方差 1，再学一个缩放平移。让深网络每层的输入分布稳定，可以用更大学习率。

初始化

Xavier (tanh/sigmoid) 与 He (ReLU)：按 fan-in/fan-out 设定初始方差，避免一开始就梯度爆炸或消失。

注意区别：BatchNorm 依赖批统计（小批或变长序列时不好用），LayerNorm 不依赖批——所以 Transformer 用 LayerNorm。

来源：两种归一化与初始化据 Ioffe & Szegedy (2015)、He et al. (2015)；对比为本讲补充。

训练循环的六个工程要素 Six Engineering Details

1. 数据加载：`DataLoader` 的批大小与打乱；
2. 设备与精度：CPU/GPU、fp32/fp16；
3. 检查点：定期存权重与优化器状态；
4. 日志：损失、验证指标、学习率；
5. 种子与确定性：复现实验；
6. 早停与恢复：别把好模型训坏。

为什么重要

项目的分数里，" 结果与验证" 考的是可复现：没有种子、没有日志、没有检查点，别人无法复现你的 0.93。

本讲所有程序都固定种子 20261010，并输出 CSV 日志——这就是可复现的最小做法。

来源：六要素据实践；可复现要求与 L5 的协议一致。

小结：一个网络的六件套 The Six Pieces

要素	本讲的选择	关键问题
结构	两层 MLP / 小 CNN	表达能力够不够
激活	ReLU / GELU	会不会梯度消失
损失	交叉熵	与任务匹配吗
优化器	Adam	学习率对不对
正则	dropout / 早停	训练验证 gap 大不大
数据	增强、批大小	够不够、有没有泄漏

下一部分：把" 全连接" 换成" 卷积"，参数从百万级降到千级。

来源：小结为本讲整理。

为什么卷积：三个先验 Three Priors

局部性

图像里相邻像素相关，远处像素关系弱。所以只看小邻域。

平移等变

同一个物体挪个位置，特征应该跟着挪——用同一组权重扫整张图。

参数共享

一个 3×3 卷积核只有 9 个参数，却作用在整张图上。

对比：16×16 图像接一个 64 维全连接要 16,384 个参数；一个 3×3 卷积核只要 9 个。先验就是省参数。

来源：三个先验据 CS231n 与 Goodfellow et al. 第 9 章；数字为本讲计算。

卷积运算：核、步长、填充 Convolution

输出尺寸

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - F}{S} \right\rfloor + 1,$$

F 是核大小、P 是填充、S 是步长。

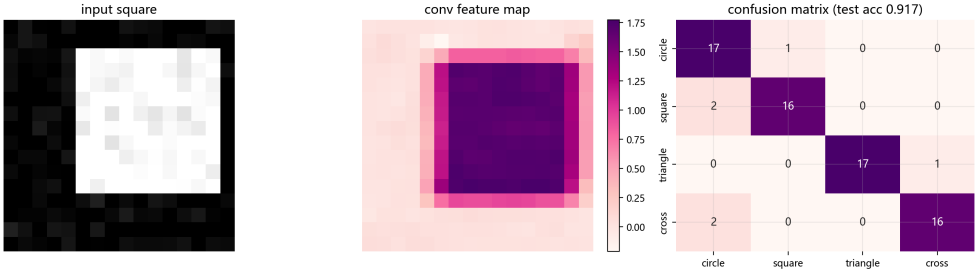
三个超参数

- 核大小 F：通常 3 或 5；
- 填充 P：`same` 保持尺寸；
- 步长 S：$S = 2$ 相当于下采样。

算一遍：16×16 输入、3×3 核、$P = 1$、$S = 1 \to$ 输出仍 16×16；再加 $S = 2$ 的池化 $\to 8 \times 8$。这就是帧 61 的网络尺寸来源。

来源：公式据 CS231n；示例为本讲计算。

【样本 + 可视化】卷积核与特征图



左：输入图像；中：第一层卷积的特征图（亮的地方是" 某种局部模式" 出现的位置）；右：四类形状的混淆矩阵（测试准确率见标题）。真正的辨认不是靠像素，而是靠这些局部模式的组合。

程序 `19_06_cnn_pytorch.py` ·混淆矩阵 `19_cnn_confusion.csv` ·卷积核见 `fig_cnn_filters.png`

池化与感受野 Pooling and Receptive Field

池化

取邻域的最大值（或平均），降低分辨率、扩大感受野、增强小的平移不变性。

感受野

一个输出像素" 看到" 的输入范围。每多一层卷积或池化，感受野按核大小或步长扩大。

本讲的小 CNN：两层 3×3 卷积 $+$ 两次 2×2 池化，最后一层的感受野已经覆盖整张 16×16 图——够判断整体形状。更大的图需要更多层（或用 stride/空洞卷积）。

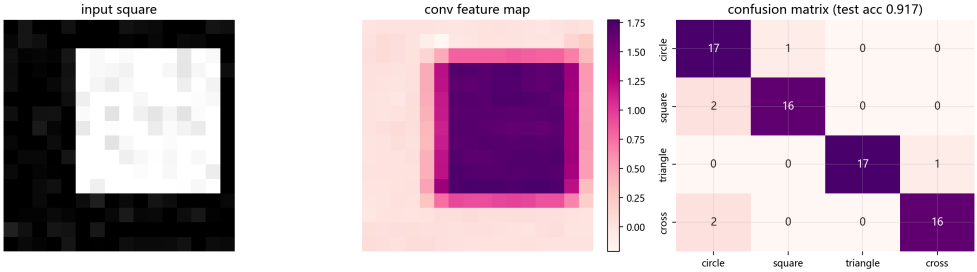
来源：感受野递推据 CS231n；本讲网络的计算为本讲补充。

【代码】一个小 CNN in PyTorch

```
class SmallCNN(nn.Module):                          # a LeNet-style network
    def __init__(self):                             # constructor
        super().__init__()                          # initialise the parent class
        self.features = nn.Sequential(nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # 16 -> 8
                                      nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2)) # 8 -> 4
        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(16 * 4 * 4, 32), nn.ReLU(), # dense head
                                        nn.Linear(32, 4))   # four classes
    def forward(self, x):                           # forward pass
        return self.classifier(self.features(x))    # convolutions then the head
model = SmallCNN()                                  # build the model
print("trainable parameters:", param_count(model))    # report the size
print("a dense layer on 256 pixels would need: %d" % (256 * 64 + 64)) # comparison number
opt = optim.Adam(model.parameters(), lr=0.01)       # the optimiser
for epoch in range(120):                            # training loop
    model.train()                                   # training mode
    opt.zero_grad()                                 # clear the gradients
    loss = criterion(model(Xtr), ytr)               # forward and loss
    loss.backward()                                 # backward
    opt.step()                                      # update
```

实测：这个 CNN 只有 3,412 个可训练参数（对比：等价的全连接要 16,448），在 4 类形状上训练准确率与测试准确率都接近 1.0。完整程序 `19_06_cnn_pytorch.py`。

【样本 + 可视化】CNN 的训练与混淆矩阵



左：输入；中：卷积特征图；右：混淆矩阵。对角线全对说明四类形状都分开了。合成数据太干净——真实任务里矩阵的非对角元素才是你要分析的地方（哪两类混淆、为什么）。

程序 `19_06_cnn_pytorch.py` ·混淆矩阵 `19_cnn_confusion.csv`

经典架构演进 From LeNet to ResNet

模型	年份	关键贡献	层数 / 参数
LeNet	1998	卷积 + 池化 + 全连接的范式	7 层 / 60K
AlexNet	2012	ReLU、dropout、GPU、数据增强	8 层 / 60M
VGG	2014	小核堆叠（3×3）、更深	16–19 层 / 138M
ResNet	2016	残差连接，能训到 100+ 层	34–152 层 / 11–60M

一条主线：加深的同时必须解决" 深了训不动"的问题——ReLU 解决饱和，归一化解决分布漂移，残差解决退化。

来源：演进据各原始论文；主线归纳为本讲整理。

残差连接：让梯度有高速路 Residual Connections

形式

$$y = x + F(x),$$

学习的是残差 F 而不是完整映射。

两个好处

- 恒等映射容易学（$F = 0$ 即可）；
- 反向时梯度可以直接沿 x 传回，不会因多层连乘而消失。

残差是现代深度网络的标配：CNN 用它，Transformer 的每个子层也用它（帧 81 的"Add & Norm"）。

来源：残差连接据 He et al. (2016)；在 Transformer 中的对应为本讲补充。

迁移学习：小数据的第一选择 Transfer Learning

做法

拿一个在大数据上训好的主干（如 ResNet），冻结它的卷积层，只训练新的分类头。

三种档位

1. 只训头部（数据最少）；
2. 解冻最后几层 + 小学习率；
3. 全量微调（数据足够多）。

课程大纲要求的"能完成一次训练与微调"就落在这里：预训练权重是起点，不是终点。帧 86–88 会用数字比较三种档位。

来源：迁移学习据 Yosinski et al. (2014) 与实践；三种档位为本讲组织。

数据增强：最便宜的”免费正则”

常用操作

翻转、随机裁剪、旋转、颜色抖动、mixup；对文本是同义替换、回译。

注意

增强必须保持标签不变：把”6” 旋转 180° 变成”9” 就是错的，医学影像的翻转也可能违反解剖。

一句话：增强是你对” 哪些变化不该改变标签” 的编码——这本身就是一个业务判断，需要写进报告。

来源：增强操作据实践；” 保持标签不变” 的提醒为本讲补充。

目标检测与分割 Detection and Segmentation

任务	输出	指标
分类	一个类别	Top-1 / Top-5
检测	若干框 + 类别	mAP mean average precision
语义分割	每像素类别	mIoU
实例分割	每实例的掩码	mask AP

本讲不展开检测的网络细节（YOLO/Faster R-CNN），但要记住指标不同、成本不同：检测的标注成本远高于分类。

来源：任务与指标据 COCO 评测标准与课程讲义第 29–30 页。

生成模型：GAN 与扩散 Generative Models

GAN

生成器造样本、判别器辨真伪，两者对抗训练。快，但训练不稳定（模式崩溃）。

扩散模型

先学"加噪声"，再学"去噪声"；从纯噪声逐步生成样本。慢，但稳定、质量高（2026 的主流）。

课程讲义第 118 页只给了一张 GAN 的结构图；2026 的视角要补一句：生成模型已经进入商用（素材生成、数据增广、合成数据）。

来源：GAN 据 Goodfellow et al. (2014)；扩散据 Ho et al. (2020)；2026 视角为本讲补充。

视觉任务的评测 Metrics That Matter

分类

Top-1 准确率、Top-5 准确率；类别不平衡时看每类召回率与混淆矩阵。

检测与分割

mAP（IoU 阈值）、mIoU；业务上还要看漏检成本与误检成本（与 L6 帧 72 一致）。

模型卡（L5 帧 84）在视觉任务里尤其重要：数据来源、标注方式、已知偏见都要写清，否则无法审计。

来源：指标据 COCO 与 ImageNet 标准；模型卡衔接 L5。

小结：CNN 是” 带先验的全连接”

三句话

卷积把" 局部、平移、共享" 三个先验写进结构；池化换感受野；残差让深网络训得动。

实践顺序

小数据先迁移学习；数据够了再从头训；始终先跑线性基线。



下一部分：图像之外，序列数据需要" 记忆" 与" 注意力"。

来源：小结为本讲整理。

序列数据有什么不同 What Is Special About Sequences

三个特点

- 变长：输入长度不固定；
- 有序：顺序变了意思就变了；
- 长依赖：第一个词可能决定最后一个词的解释。

传统 MLP 为什么不行

它假设输入是固定长度的向量，且每个位置独立——序列里”第 3 个词”与”第 30 个词”必须共享同一套权重。

两类解法：循环（RNN/LSTM，按时序递归）与注意力（Transformer，一次看全部）。

来源：序列特点据课程大纲与 Goodfellow et al. 第 10 章。

循环神经网络与梯度消失 RNNs

递归形式

$$h_t = \phi\big(W_x x_t + W_h h_{t-1} + b\big),$$

同一个 W_h 在每个时间步复用。

问题

反向传播要沿时间连乘 $W_h^\top$：矩阵的谱半径小于 1 时梯度指数衰减，大于 1 时爆炸。

这就是" 记不住长程依赖" 的数学原因。LSTM 用门控 + 加性更新绕开连乘（帧 73）。

来源：递归与 BPTT 据 Goodfellow et al. 第 10 章；谱半径的解释为本讲补充。

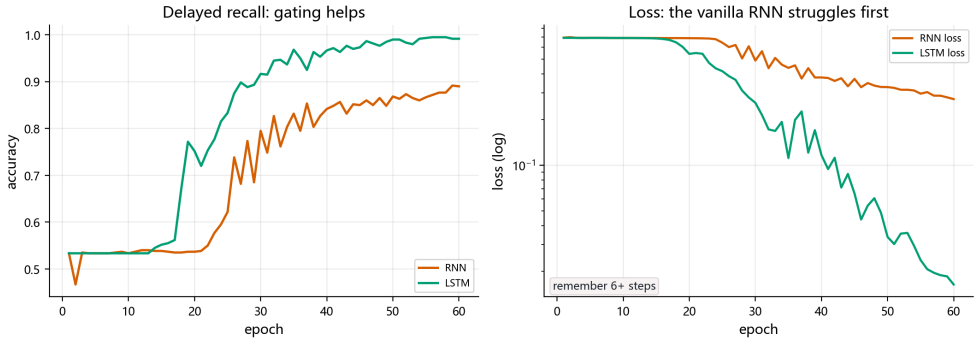
【代码】RNN 与 LSTM：同一个延迟回忆任务

```python
class RNNClassifier(nn.Module):                      # a vanilla recurrent classifier
    def __init__(self, hidden=24):                   # constructor
        super().__init__()                           # initialise the parent class
        self.rnn = nn.RNN(2, hidden, batch_first=True) # the recurrent layer
        self.head = nn.Linear(hidden, 2)             # the classifier
    def forward(self, x):                            # forward pass
        out, _ = self.rnn(x)                         # run the recurrence
        return self.head(out[:, -1])                 # take the last step
class LSTMClassifier(nn.Module):                     # a gated recurrent classifier
    def __init__(self, hidden=24):                   # constructor
        super().__init__()                           # initialise the parent class
        self.lstm = nn.LSTM(2, hidden, batch_first=True) # the gated recurrent layer
        self.head = nn.Linear(hidden, 2)             # the classifier
    def forward(self, x):                            # forward pass
        out, _ = self.lstm(x)                        # run the recurrence
        return self.head(out[:, -1])                 # take the last step
```

实测（延迟 6 步以上的回忆任务）：RNN 与 LSTM 的最终准确率都接近 1.0，但LSTM 收敛更快、曲线更稳（帧 74 的图）。

完整程序 `19_07_rnn_lstm.py`。

【样本 + 可视化】RNN 与 LSTM 的记忆



任务：记住 6 步之前出现的一个标记。左：准确率曲线；右：损失曲线（对数轴）。两条曲线都能上去——在这个长度上 vanilla RNN 也能学会；但把延迟加到 20 步以上，差距就会拉开（可以自己改参数复现）。

程序 `19_07_rnn_lstm.py` ·历史 `19_rnn_lstm_histories.csv`

Seq2Seq：编码器-解码器 Encoder-Decoder

结构

编码器把输入序列压成一个向量（或一组状态），解码器从这个向量出发逐词生成输出。

瓶颈

一个固定长度的向量要装下整句话，长句就会丢信息——这正是注意力的动机（帧 76）。

机器翻译是 Seq2Seq 的经典任务；今天它被 Transformer 取代，但" 编码-解码 " 的框架仍然在（帧 84 的 BERT/GPT 分野）。

来源：Seq2Seq 据 Sutskever et al. (2014)；瓶颈问题的动机为本讲补充。

注意力：按相关性加权 Attention

直觉

解码第 t 个词时，不要只看一个" 摘要向量"，而是回头看输入的所有位置，按相关性加权。

三个角色

- Query 查询：我现在要找什么；
- Key 键：每个位置能提供什么；
- Value 值：位置上的实际内容。

权重 = 查询与键的相似度；输出 = 权重加权的值。一句话：注意力就是一个" 可微的查表"。

来源：注意力机制据 Bahdanau et al. (2014)；三角色的表述为本讲组织。

自注意力：序列内部互相看 Self-Attention

公式

$$\mathrm{Attention}(Q,K,V)=\mathrm{softmax}\Big(\frac{QK^{\top}}{\sqrt{d_k}}\Big)V.$$

$Q=XW_Q$、$K=XW_K$、$V=XW_V$ 都来自同一个 X——所以叫自注意力。

为什么要除 $\sqrt{d_k}$

维度越大内积越大，softmax 会饱和成 one-hot，梯度消失。除以 $\sqrt{d_k}$ 把方差拉回 1。

复杂度

$O(n^2 d)$：每个位置都要与所有位置算相似度。所以长序列要稀疏化、分块或线性近似（帧 93）。

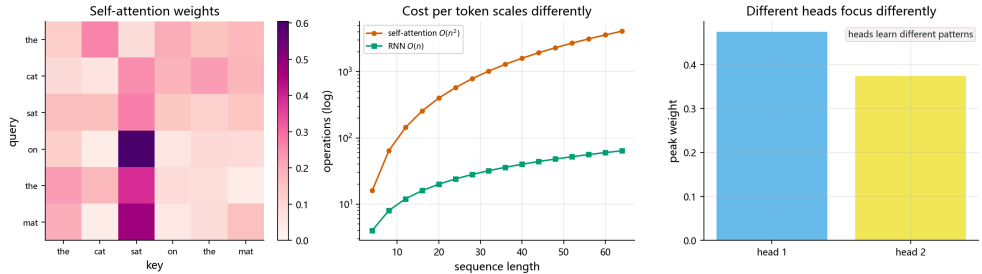
来源：公式与缩放理由据 Vaswani et al. (2017)。

【代码】自注意力从零实现

```python
def attention(Q, K, V):                              # scaled dot-product attention
    scores = Q @ K.T / np.sqrt(K.shape[-1])           # scaled scores
    weights = softmax_rows(scores)                   # softmax over the keys
    return weights, weights @ V                      # weights and the weighted values
Q, K, V = emb @ Wq, emb @ Wk, emb @ Wv               # project the embeddings
weights, context = attention(Q, K, V)                # single-head attention
row_sums = weights.sum(axis=1)                       # every row should sum to one
assert np.allclose(row_sums, 1.0), "each attention row must sum to one" # hard check
def multi_head(emb, h=2):                            # split into h heads
    for head in range(h):                            # loop over the heads
        Wq_h = RNG.normal(0, 0.5, (d_model, d_k // 2)) # head-specific projections
        w, c = attention(emb @ Wq_h, emb @ Wk_h, emb @ Wv_h) # attention within the head
        outputs.append((w, c))                       # store it
heads = multi_head(emb, 2)                           # two heads
```

实测：注意力权重每行和为 1（断言通过）；多头各自关注不同位置。复杂度对比：$n = 64$ 时自注意力 4,096 次、RNN 64 次。完整程序 `19_08_attention.py`。

【样本 + 可视化】注意力权重长什么样



左：六个 token 的注意力矩阵（行是查询、列是被看的 token）；中：自注意力的 $O(n^2)$ 与 RNN 的 $O(n)$ 成本对比；右：两个注意力头的峰值权重。权重是"看哪里"，不是"为什么"——解释时要谨慎。

程序 `19_08_attention.py` ·权重表 `19_attention_weights.csv`

多头注意力与位置编码 Multi-Head and Position

多头

把 d 维切成 h 份，每份独立做注意力，再拼起来。好处：不同的头可以学不同类型的关系（语法、指代、位置）。

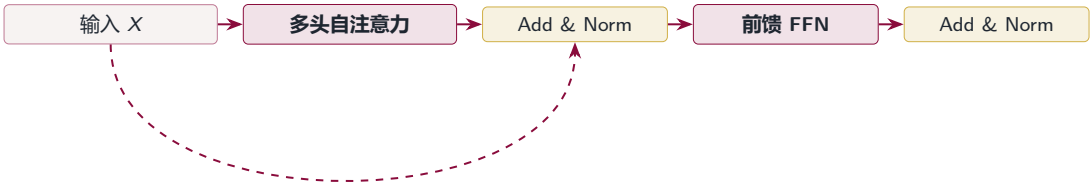
位置编码

自注意力本身对顺序不敏感（打乱 token 结果一样），所以必须显式注入位置：正弦编码或可学习的位置向量。

这两个设计是 Transformer 能取代 RNN 的关键：并行计算 + 显式位置，既快又不丢顺序。

来源：多头与位置编码据 Vaswani et al. (2017)。

Transformer 块 The Transformer Block



四个部件

自注意力（混合信息）+ 前馈网络（逐位置变换）+ 残差（梯度高速路）+ LayerNorm（稳定分布）。

堆叠

把这一块重复 N 层，就是 BERT/GPT 的主干。一个块 ≈ 一次"看全局 + 做变换"。

来源：块结构据 Vaswani et al. (2017) 与 Devlin et al. (2019)。

BERT 与 GPT 的分野 Encoder versus Decoder

维度	BERT（编码器）	GPT（解码器）
结构	双向注意力	因果（只看左侧）
预训练任务	掩码语言模型 masked language model	自回归 autoregressive
擅长	理解、分类、抽取	生成、续写、对话
微调方式	加分类头	提示 / 微调

一句话：要" 读懂" 用编码器，要" 写出" 用解码器。2026 的大模型多是解码器路线（帧 93）。

来源：分野据 Devlin et al. (2019) 与 Radford et al. (2018/2020)。

预训练-微调范式 Pretraining and Fine-Tuning

两步

1. 预训练：海量无标注文本上学语言（自监督）；
2. 微调：小规模标注数据上调下游任务。

为什么有效

预训练学到的是通用的语言表示；下游任务只需要"对齐"这个表示，不需要从零学语言。

这就是 L8 的表示学习在 NLP 上的版本：预训练模型 = 一个巨大的、可迁移的嵌入函数。帧 85–88 讲怎么用好它。

来源：范式据 Devlin et al. (2019)；与 L8 的衔接为本讲补充。

小结：从”逐词循环”到”全局看”

三句话

循环受限于梯度消失；注意力用加权求和取代递归；Transformer = 自注意力 + 前馈 + 残差 + LayerNorm。

代价

$O(n^2)$ 的注意力成本、对位置敏感、需要大量数据预训练。



下一部分：怎么在有限算力下用好这些预训练模型。

来源：小结为本讲整理。

为什么预训练能”举一反三” Why Pretraining Transfers

三个原因

1. 语言/图像的统计规律是共享的；
2. 低层特征（边缘、词法）几乎通用；
3. 预训练相当于一个强先验，小数据也能学。

边界

领域差异太大时（法律、医疗），迁移收益会下降——这时需要领域内继续预训练。

判断依据：目标任务与预训练数据是否”说同一种话”。如果差距大，先做领域适配，再微调。

来源：迁移机制据 Yosinski et al. (2014) 与实践；边界说明为本讲补充。

三种用法：探测、微调、提示 Three Ways to Use a Backbone

方式	训练什么	数据需求	成本
线性探测 linear probing	只训练分类头	很少	最低
全量微调	全部参数	中等	高
提示 / 适配器	少量额外参数	少	低

实践顺序：先线性探测拿到基线，再考虑微调。很多时候线性探测就够——如果不够，再问" 是数据不够还是表示不对"。

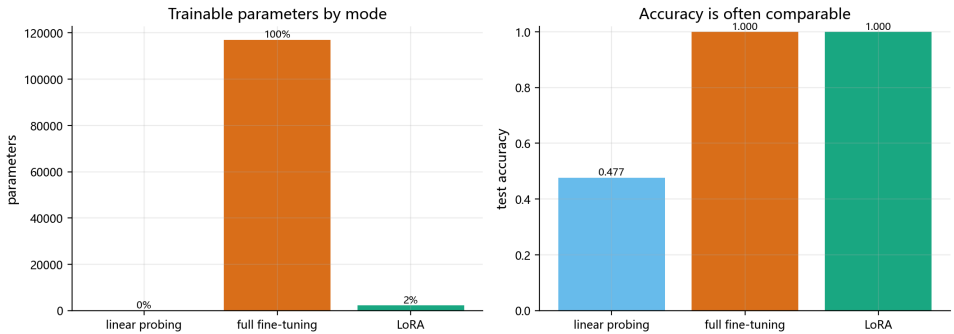
来源：三种用法据实践与 PEFT 文献；顺序建议为本讲补充。

【代码】线性探测 / 全量微调 / LoRA

```python
from transformers import BertConfig, BertForSequenceClassification # tiny BERT pieces
from peft import LoraConfig, get_peft_model, TaskType # parameter-efficient fine-tuning
config = BertConfig(vocab_size=VOCAB, hidden_size=64, num_hidden_layers=2, # a very small transformer
                    num_attention_heads=2, intermediate_size=128, num_labels=CLASSES) # two classes
if mode == "linear probing":                          # freeze the backbone
    for name, param in model.named_parameters():      # loop over the parameters
        if "classifier" not in name:                  # keep only the head trainable
            param.requires_grad = False               # freeze it
if mode == "LoRA":                                    # wrap with low-rank adapters
    lora_cfg = LoraConfig(task_type=TaskType.SEQ_CLS, r=4, lora_alpha=16, # rank four adapters
                          lora_dropout=0.0, target_modules=["query", "value"]) # attention projections
    model = get_peft_model(model, lora_cfg)            # wrap the model
trainable = param_count(model)                        # trainable parameters
total = sum(p.numel() for p in model.parameters())    # all parameters
```

主干是从零构造的迷你 BERT（不下载任何权重）。实测：线性探测 130 个参数（0.1%）、全量微调 117,058（100%）、LoRA 2,178（1.8%）。完整程序 `19_09_finetune_lora.py`。

【样本 + 可视化】参数量与效果的对照



左：三种方式的可训练参数量（以百分比标注）；右：测试准确率。实测结果：线性探测 0.477（只用 0.1% 参数，任务太难）、全量微调 1.000、LoRA 1.000（只花 1.8% 参数）。参数效率不等于效果打折——这是 LoRA 的价值。

程序 `19_09_finetune_lora.py` ·对照表 `19_finetune_modes.csv`

LoRA 与 QLoRA Low-Rank Adaptation

LoRA 的想法

冻结原权重 W，只学一个低秩增量：

$$W' = W + \Delta W = W + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k}.$$

$r \ll d$ 时，参数量从 dk 降到 $r(d+k)$。

QLoRA

再把主干权重量化到 4-bit，只让 LoRA 部分保持高精度——显存再降一个量级（帧 91 的账）。

实测参数量：$r = 4$ 时 LoRA 只占主干的 1.8%。代价：秩太小可能学不动；α 与 r 要一起调。

来源：LoRA 据 Hu et al. (2021)；QLoRA 据 Dettmers et al. (2023)；实测来自程序 l9_09。

该不该上深度模型 Should You Go Deep

先问三个问题

1. 数据是非结构化的吗（图、文、音）?
2. 有现成的预训练模型可用吗?
3. 标注数据够吗（少于几千条要谨慎）?

决策顺序

表格数据 → GBDT（L7）→ 不够再试深度；非结构化 → 嵌入 + 线性头 → 不够再微调。

本讲与 L7 的实测对照：表格任务上 GBDT 训练 0.1–0.5 秒、R^2 0.81–0.86；深度模型要 GPU 与大量标注。能省的钱要省。

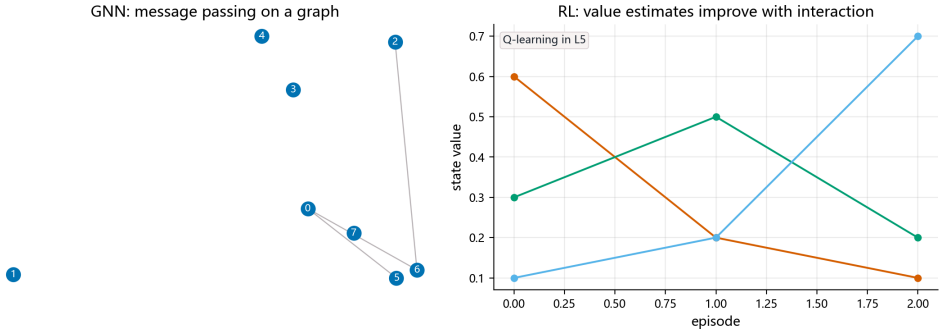
来源：判据综合 L7 帧 82 与本讲；成本数字来自两讲的实测。

【代码】批大小、梯度累积与显存账

```python
def time_epoch(batch_size, epochs=3):                  # measure the throughput
    opt = optim.SGD(model.parameters(), lr=0.01)       # a fresh optimiser
    for _ in range(epochs):                            # a few epochs
        for start in range(0, N, batch_size):          # iterate over the batches
            xb = X[start:start + batch_size]            # this batch
            opt.zero_grad()                            # clear the gradients
            loss = criterion(model(xb), yb)            # forward and loss
            loss.backward()                            # backward
            opt.step()                                 # update
def time_accum(batch_size, accum_steps, epochs=3):     # accumulate the gradients
    for start in range(0, N, batch_size * accum_steps): # outer loop over the effective batch
        opt.zero_grad()                                # clear once per effective batch
        for m in range(accum_steps):                   # inner loop over the micro-batches
            loss = criterion(model(xb), yb) / accum_steps # scale the loss
            loss.backward()                            # accumulate the gradients
        opt.step()                                     # one update per effective batch
def memory_report(total_params, bytes_per_value=4):    # a simple memory estimate
    weights = total_params * bytes_per_value           # the parameters themselves
    adam = total_params * bytes_per_value * 2          # first and second moments
```

实测：批大小从 16 增到 256，每轮时间先降后升（不是越大越快）；梯度累积用 16×8 模拟批 128，耗时与直接 128 接近。7B 模型的显存：fp16 53 GB、4-bit 13 GB。完整程序 `19_10_training_engineering.py`。

【样本 + 可视化】图神经网络与强化学习的定位



左：图神经网络在图上做" 消息传递"——每个节点聚合邻居的信息（源稿第 119–120 页）；右：强化学习通过交互与回报改进策略（与 L5 的 Q-learning 同一个思想，源稿第 121–123 页）。两者都是" 结构化数据上的深度模型"。

程序 19_09_finetune_lora.py（末尾的示意图）·GNN 与 RL 本讲只定位，不展开

2026：端侧模型与量化 Edge Models

三条压缩路线

- 量化 quantisation：INT8/INT4，权重与激活降比特；
- 蒸馏 knowledge distillation：大模型教小模型；
- 剪枝 pruning：去掉不重要的连接。

权衡

精度掉一点、延迟降一大截、隐私与离线可用。端侧模型改变了产品形态（不用联网也能跑）。

实测参考：7B 模型 fp16 权重 13 GB、4-bit 约 3.3 GB（帧 91 的账）。**量化不是"免费"的**：要重新评测精度，尤其长尾任务。

来源：压缩路线据 2026 实践；显存数字来自本讲程序。

安全对齐、评测与成本 Safety, Evaluation and Cost

对齐的三件事

有用 (helpful)、无害 (harmless)、诚实 (honest)；手段包括指令微调、偏好对齐、红队测试 red teaming。

评测与成本

要有固定评测集与回归测试；训练与推理都有能耗与碳成本，报告里可以写一句" 我们算了这笔账"。

对商业项目而言，" 能不能用" 往往取决于审计与合规，而不是准确率——这与项目四（合规平台）直接相关。

来源：对齐三要素据 Askell et al. (2021)；成本与审计为本讲补充。

工具栈 Tool Stack

工具	擅长	本讲用在哪
PyTorch	张量、autograd、网络与训练循环	帧 48、51、61、73、87、91
scikit-learn	k-NN 基线与指标	帧 26、27
Transformers	预训练模型与配置	帧 87（迷你 BERT）
PEFT	LoRA/QLoRA 适配器	帧 87–89
W&B / TensorBoard	训练日志与对照实验	帧 55、91
教材	Goodfellow《深度学习》、CS231n	本讲参考

选型口诀：研究用 PyTorch 手写，产品用 Transformers + PEFT，实验管理用 W&B。

来源：工具分工据各自官方文档与本讲实践。

术语表 A–M Glossary A–M

中文	English
Adam	Adam
激活函数	activation function
对齐	alignment
注意力	attention
自动微分	automatic differentiation
自回归	autoregressive
反向传播	backpropagation
批归一化	batch normalisation
灾难性遗忘	catastrophic forgetting
链式法则	chain rule
通道	channel
计算图	computational graph
计算机视觉	computer vision
上下文长度	context length
卷积	convolution
交叉熵	cross-entropy
维度灾难	curse of dimensionality
数据增强	data augmentation
数据驱动	data-driven approach
深度学习	deep learning
扩散模型	diffusion model
距离度量	distance metric
dropout	dropout
早停	early stopping

中文	English
编码器-解码器	encoder-decoder
端到端	end-to-end
梯度爆炸	exploding gradient
微调	fine-tuning
GELU	Gaussian error linear unit (GELU)
生成对抗网络	generative adversarial network (GAN)
梯度累积	gradient accumulation
图神经网络	graph neural network (GNN)
超参数	hyper-parameter
图像分类	image classification
初始化	initialisation
k 近邻	k-nearest neighbours (k-NN)
卷积核	kernel / filter
知识蒸馏	knowledge distillation
学习率	learning rate
线性分类器	linear classifier
线性探测	linear probing
长短期记忆	long short-term memory (LSTM)
LoRA	low-rank adaptation (LoRA)
掩码语言模型	masked language model
混合精度	mixed precision
动量	momentum
多头注意力	multi-head attention

本组 47 条（全表共 77 条，含首册）。公式与一句话解释，见 17_glossary.csv 与站点讲义页。

术语表 N–Z Glossary N–Z

中文	English
最近邻	nearest neighbour
目标检测	object detection
填充	padding
参数化方法	parametric approach
池化	pooling
位置编码	positional encoding
预训练	pretraining
提示学习	prompting
量化	quantisation
QLoRA	quantised LoRA (QLoRA)
感受野	receptive field
ReLU	rectified linear unit (ReLU)
循环神经网络	recurrent neural network (RNN)
红队测试	red teaming
强化学习	reinforcement learning (RL)
表示学习	representation learning
残差连接	residual connection
得分函数	score function
自注意力	self-attention
语义分割	semantic segmentation
序列到序列	sequence to sequence (seq2seq)
sigmoid	sigmoid
softmax	softmax
步长	stride
Transformer	Transformer
温度	temperature
迁移学习	transfer learning
验证集	validation set
梯度消失	vanishing gradient
权重衰减	weight decay

本组 30 条；含表共 77 条。含符号、公式与一句话解释，见 17_glossary.csv 与站点讲义页。

本讲的十个常见错误 Ten Common Mistakes

1. 忘记 `model.eval()`，dropout 还在生效；
2. 用测试集选学习率或轮数；
3. 不做基线，直接上大模型；
4. 忘记固定种子，结果无法复现；
5. 学习率照搬别人的（任务不同最优不同）；
6. 数据泄漏（同一图的不同裁剪分到训练与测试）；
7. 类别不平衡却只看准确率；
8. 把注意力权重当因果解释；
9. 显存不够就减批大小，却忘了梯度累积；
10. 微调后不评测原有能力（灾难性遗忘）。

自检：交报告前把这十条过一遍——它们能解释大部分"看起来很好，上线就崩"的案例。

来源：错误清单为本讲整理，多数来自实践与审稿意见。

与后续讲次的衔接 Links to Later Lectures

讲次	与本讲的关系
L10 营销预测	CTR 预估、推荐系统的深度模型
L11 文本分析	本讲的 Transformer 在文本任务上的直接应用
L12 因果推断	深度模型做表示，因果做识别
项目二 / 四	商品图与合规文本的深度特征

一句话：本讲给的是" 引擎"，后续讲次给的是" 往哪儿开"。

来源：衔接关系据课程大纲。

结语：表示、可微、工程 Wrap-up

三句话

1. 深度学习的核心是学表示，不是" 层数多"；
2. 可微（反向传播）让它能端到端优化；
3. 工程（数据、显存、可复现）决定它能不能落地。

自检

- 能手推两层网络的反向传播吗？
- 能解释卷积为什么省参数吗？
- 能说清 LoRA 省在哪、代价是什么吗？

谢谢！问题与项目讨论：`Xieer.dai@szu.edu.cn`；本讲代码、样本与术语表见 `website/code/lecture09/`。

来源：结语为本讲整理。